

# **Unit IV Activity II**

Time Series, Dose-Response Curves & Decomposition — Code and Output

10 Questions — R and Python as specified per question

Date: 12/08/2026

## Q1

# Daily Sales Time Series (2 Years)

### Code (R)

```
library(ggplot2)

# Simulated dataset: daily sales over 2 years (730 days), with a gradual
# upward trend plus day-to-day noise and mild weekly seasonality.
set.seed(1)
n_days <- 730
dates <- seq(as.Date("2024-01-01"), by="day", length.out=n_days)
trend <- seq(8000, 11500, length.out=n_days)
weekly_effect <- rep(c(0, -200, -300, -100, 200, 800, 600), length.out=n_days)
noise <- rnorm(n_days, 0, 350)
sales <- trend + weekly_effect + noise
df <- data.frame(Date=dates, Sales=sales)

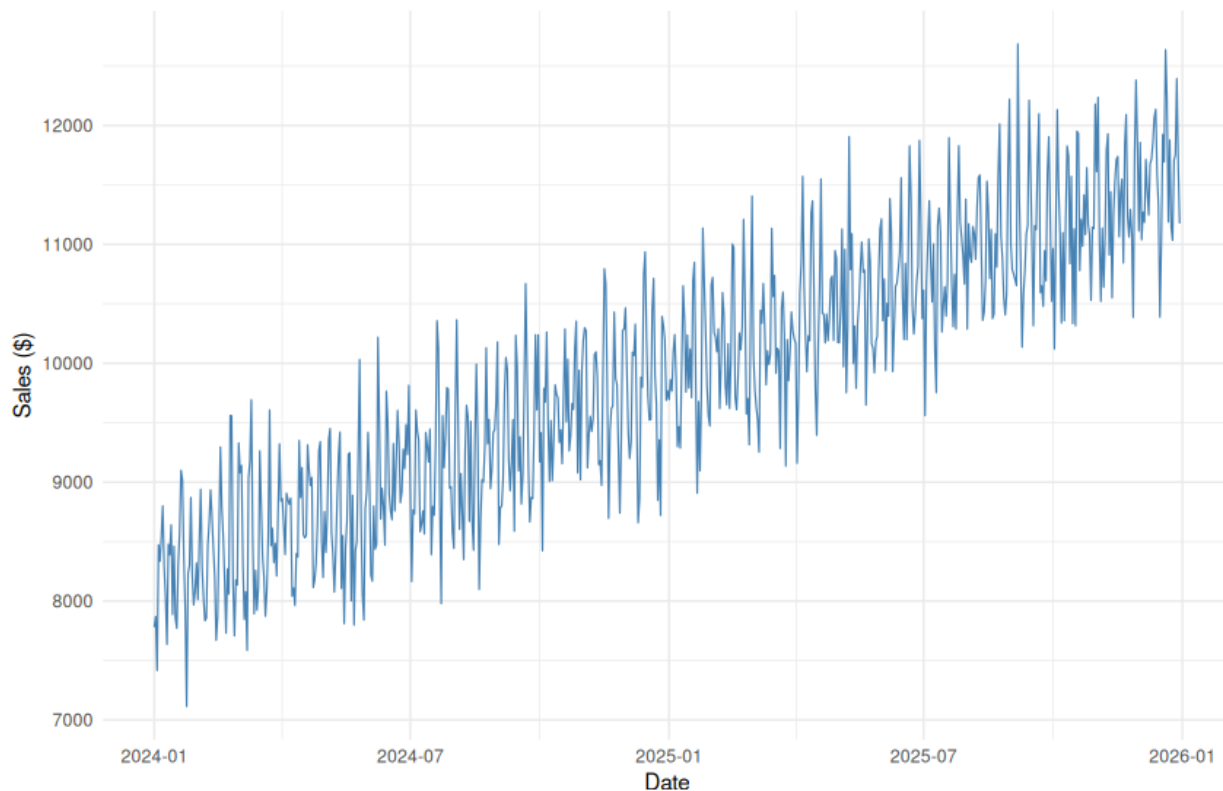
p <- ggplot(df, aes(x=Date, y=Sales)) +
  geom_line(color="steelblue", linewidth=0.4) +
  labs(title="Daily Sales Over the Past Two Years", x="Date", y="Sales ($)") +
  theme_minimal()
print(p)

# Identify long-term trend via simple linear regression on day index
fit <- lm(Sales ~ as.numeric(Date), data=df)
slope <- coef(fit)[2]
cat(sprintf("Estimated trend: %.2f $ change per day (%.0f $ per year)\n", slope, slope*365))

# Interpretation: the regression slope is positive, confirming a clear
# long-term UPWARD movement in daily sales over the two-year window - daily
# noise and weekly dips/peaks are present, but the underlying trend line
# rises steadily from roughly $8,000/day to $11,500/day.
```

### Output

Daily Sales Over the Past Two Years



## Q2

# Server Temperature — Overheating Highlighted

### Code (Python)

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Simulated dataset: hourly server temperature over 5 days (120 hours),
# with a daily heat cycle plus occasional overheating spikes.
np.random.seed(2)
n_hours = 120
hours = pd.date_range("2026-01-01", periods=n_hours, freq='h')
daily_cycle = 5 * np.sin(np.linspace(0, 2 * np.pi * 5, n_hours))
base_temp = 65 + daily_cycle + np.random.normal(0, 1.5, n_hours)
# inject overheating spikes around hours 30-34 and 80-85
base_temp[30:35] += 15
base_temp[80:86] += 12
temperature = base_temp

SAFE_THRESHOLD = 75

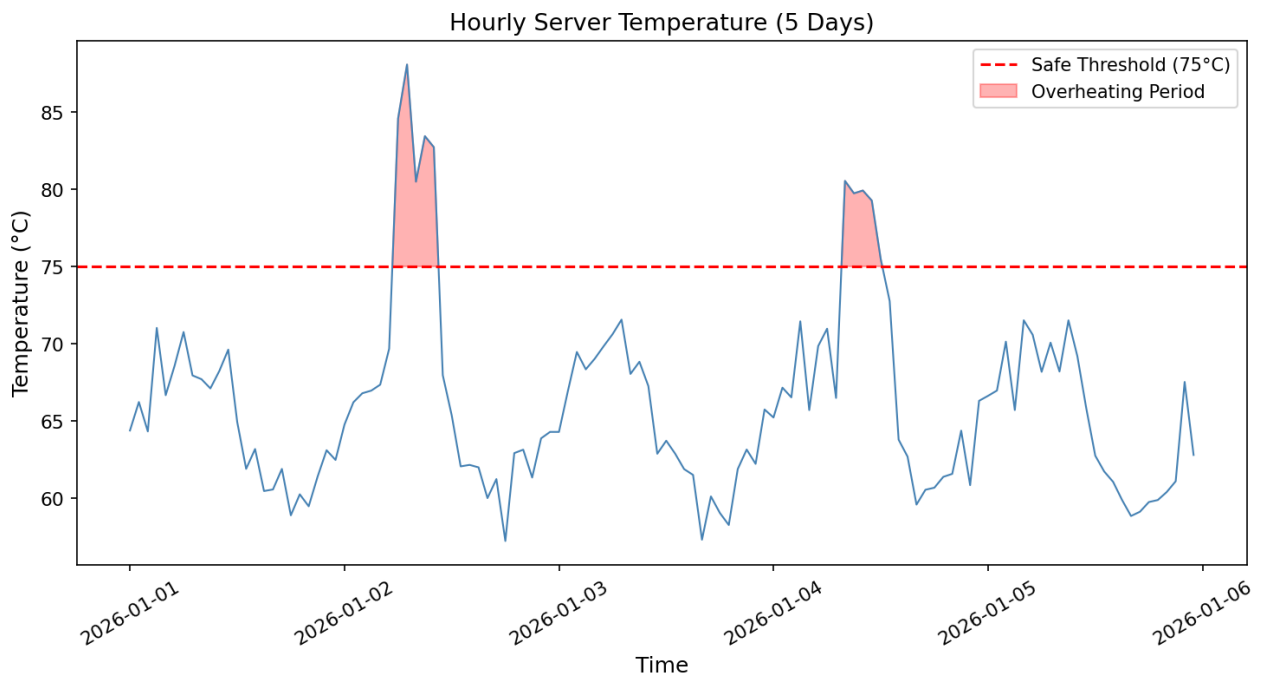
plt.figure(figsize=(10, 5.5))
plt.plot(hours, temperature, color='steelblue', linewidth=1)
plt.axhline(SAFE_THRESHOLD, color='red', linestyle='--', label=f'Safe Threshold ({SAFE_THRESHOLD}°C)')

over_mask = temperature > SAFE_THRESHOLD
plt.fill_between(hours, SAFE_THRESHOLD, temperature, where=over_mask,
                 color='red', alpha=0.3, interpolate=True, label='Overheating Period')

plt.title("Hourly Server Temperature (5 Days)")
plt.xlabel("Time"); plt.ylabel("Temperature (°C)")
plt.legend()
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

over_hours = hours[over_mask]
print(f"Number of hours exceeding safe threshold: {over_mask.sum()}")
if len(over_hours):
    print(f"First overheating period starts: {over_hours[0]}")
```

### Output



## Q3

# Three Stocks: Closing Price Comparison

### Code (R)

```
library(ggplot2)
library(reshape2)

# Simulated dataset: daily closing prices for 3 competing tech stocks over
# 1 trading year (250 days), each a random walk with different volatility.
set.seed(3)
n_days <- 250
dates <- seq(as.Date("2025-01-01"), by="day", length.out=n_days)

stock_a <- 100 + cumsum(rnorm(n_days, 0.05, 1.0)) # steadier
stock_b <- 100 + cumsum(rnorm(n_days, 0.08, 2.5)) # more volatile
stock_c <- 100 + cumsum(rnorm(n_days, 0.03, 1.6)) # mid volatility

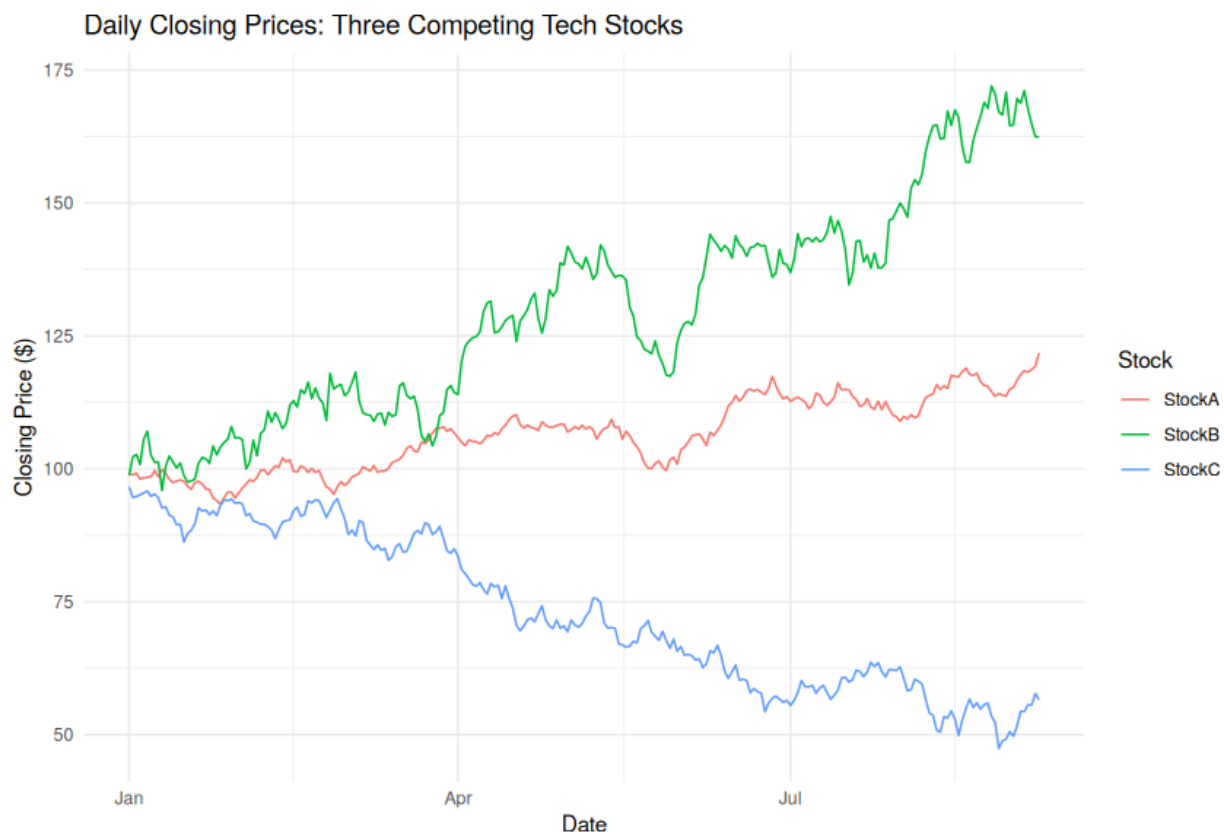
df <- data.frame(Date=dates, StockA=stock_a, StockB=stock_b, StockC=stock_c)
long_df <- melt(df, id.vars='Date', variable.name='Stock', value.name='Price')

p <- ggplot(long_df, aes(x=Date, y=Price, color=Stock)) +
  geom_line() +
  labs(title="Daily Closing Prices: Three Competing Tech Stocks",
       x="Date", y="Closing Price ($)") +
  theme_minimal()
print(p)

daily_returns <- data.frame(
  StockA = diff(stock_a),
  StockB = diff(stock_b),
  StockC = diff(stock_c)
)
vol <- sapply(daily_returns, sd)
cat("Daily price-change standard deviation (volatility) per stock:\n")
print(round(vol, 3))
cat(sprintf("\nLeast volatile: %s\n", names(which.min(vol))))

# Comment: Stock A shows the smallest day-to-day standard deviation, so it
# appears the LEAST volatile / steadiest of the three - Stock B swings the
# most sharply day to day despite a similar overall upward drift.
```

### Output



# Four Neighborhoods: Electricity Consumption

## Code (Python)

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Simulated dataset: monthly electricity consumption (kWh) across 4
# neighborhoods over 24 months, with different stability profiles.
np.random.seed(4)
n_months = 24
months = pd.date_range("2024-01-01", periods=n_months, freq='MS')

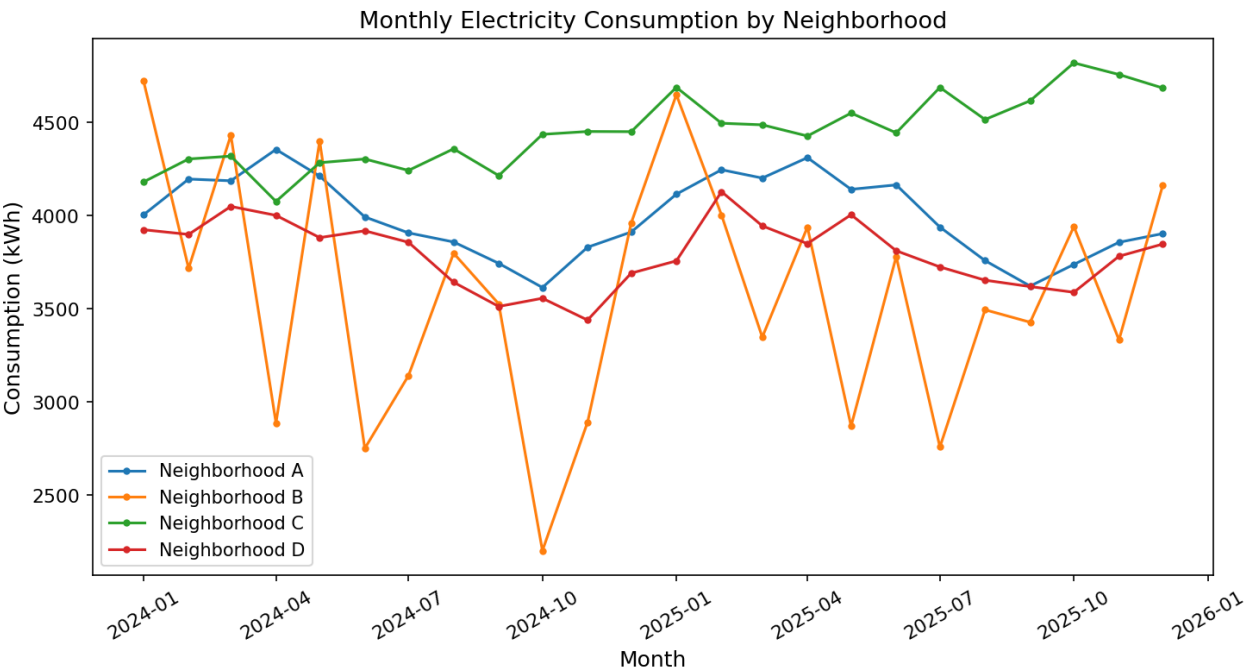
neighborhood_A = 4000 + 300 * np.sin(np.linspace(0, 4 * np.pi, n_months)) + np.random.normal(0, 80, n_months)
neighborhood_B = 3500 + np.random.normal(0, 550, n_months) # most erratic - large random swings
neighborhood_C = 4200 + 20 * np.arange(n_months) + np.random.normal(0, 100, n_months)
neighborhood_D = 3800 + 200 * np.sin(np.linspace(0, 4 * np.pi, n_months)) + np.random.normal(0, 90, n_months)

df = pd.DataFrame({'Neighborhood A': neighborhood_A, 'Neighborhood B': neighborhood_B,
                  'Neighborhood C': neighborhood_C, 'Neighborhood D': neighborhood_D}, index=months)

plt.figure(figsize=(10, 5.5))
for col in df.columns:
    plt.plot(df.index, df[col], marker='o', markersize=3, label=col)
plt.title("Monthly Electricity Consumption by Neighborhood")
plt.xlabel("Month"); plt.ylabel("Consumption (kWh)")
plt.legend()
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

volatility = df.std().sort_values(ascending=False)
print("Standard deviation of monthly consumption per neighborhood:")
print(volatility.round(1))
print(f"\nMost erratic: {volatility.index[0]}")
```

## Output



## Q5

# Dose-Response Curve — Drug Trial

### Code (R)

```
library(ggplot2)

# Simulated dataset: drug dose (mg) vs patient response (%), following a
# sigmoid (Hill-equation-like) dose-response relationship - response rises
# steeply in a mid-dose range then plateaus at high doses.
set.seed(5)
dose <- c(0, 5, 10, 20, 40, 80, 160, 320)
true_response <- 95 / (1 + exp(-(log(dose + 1) - log(30)) / 0.5))
response <- pmax(0, true_response + rnorm(length(dose), 0, 3))
df <- data.frame(Dose=dose, Response=response)

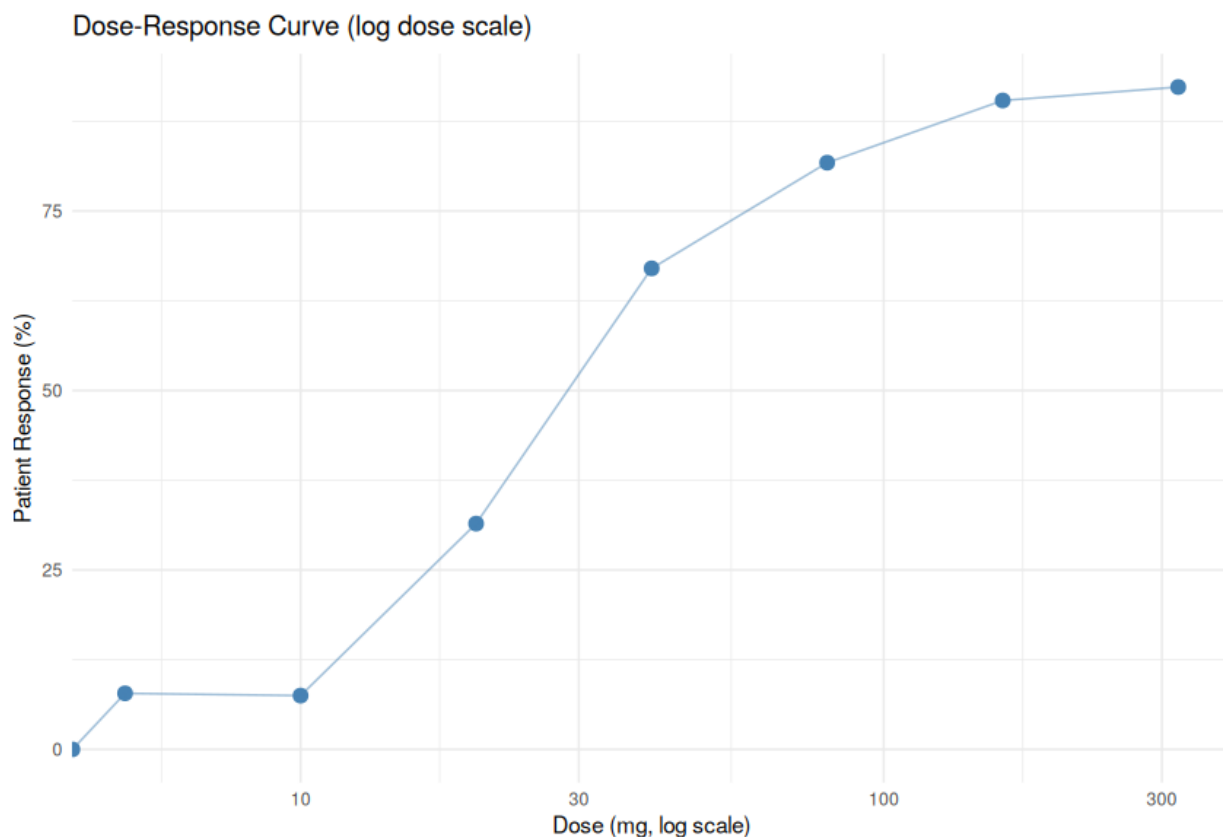
p <- ggplot(df, aes(x=Dose, y=Response)) +
  geom_point(size=3, color="steelblue") +
  geom_line(color="steelblue", alpha=0.5) +
  scale_x_log10() +
  labs(title="Dose-Response Curve (log dose scale)",
       x="Dose (mg, log scale)", y="Patient Response (%)") +
  theme_minimal()
print(p)

print(df)

# Estimate plateau dose: first dose where response reaches within 90% of
# the maximum observed response
max_resp <- max(df$Response)
plateau_dose <- df$Dose[which(df$Response >= 0.9 * max_resp)[1]]
cat(sprintf("\nMaximum response: %.1f%%\n", max_resp))
cat(sprintf("Estimated plateau dose (reaches >=90%% of max response): %d mg\n", plateau_dose))

# Interpretation: response climbs steeply between roughly 10-80 mg, then
# flattens out from about 160 mg onward - additional dose beyond this point
# yields little extra benefit (only ~2 percentage points more at 320 mg),
# suggesting ~160 mg as the minimum effective dose worth carrying forward
# into clinical trials rather than escalating further to 320 mg.
```

### Output



## Q6

# Fertilizer Dose-Response Curve — Crop Yield

### Code (Python)

```
import matplotlib.pyplot as plt
import numpy as np

# Simulated dataset: fertilizer amount (kg/hectare) vs crop yield
# (tonnes/hectare). Yield increases with fertilizer up to a point, then
# declines with over-application (a classic diminishing-returns /
# over-fertilization curve).
np.random.seed(6)
fertilizer = np.array([0, 20, 40, 60, 80, 100, 120, 140, 160, 180, 200])
true_yield = 2.0 + 0.09 * fertilizer - 0.00035 * fertilizer**2
yield_ = true_yield + np.random.normal(0, 0.15, len(fertilizer))

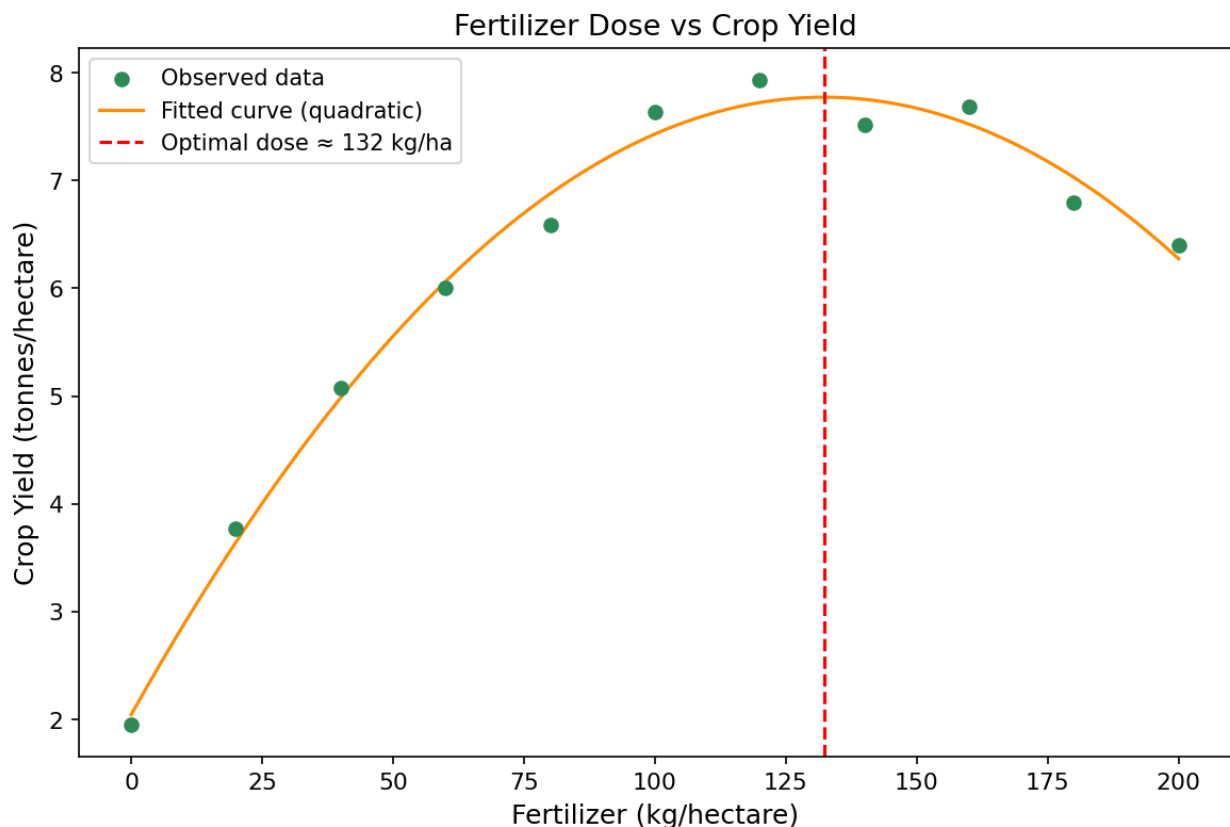
# Fit a quadratic curve (degree 2 polynomial)
coeffs = np.polyfit(fertilizer, yield_, 2)
poly = np.poly1d(coeffs)
x_smooth = np.linspace(0, 200, 300)
y_smooth = poly(x_smooth)

optimal_dose = -coeffs[1] / (2 * coeffs[0]) # vertex of the parabola
optimal_yield = poly(optimal_dose)

plt.figure(figsize=(8, 5.5))
plt.scatter(fertilizer, yield_, color='seagreen', label='Observed data', zorder=3)
plt.plot(x_smooth, y_smooth, color='darkorange', label='Fitted curve (quadratic)')
plt.axvline(optimal_dose, color='red', linestyle='--',
            label=f'Optimal dose \u2248 {optimal_dose:.0f} kg/ha')
plt.title("Fertilizer Dose vs Crop Yield")
plt.xlabel("Fertilizer (kg/hectare)"); plt.ylabel("Crop Yield (tonnes/hectare)")
plt.legend()
plt.tight_layout()
plt.show()

print(f"Optimal fertilizer dose (peak of fitted curve): {optimal_dose:.1f} kg/ha")
print(f"Predicted yield at optimal dose: {optimal_yield:.2f} tonnes/ha")
print(f"\nAdvised optimal dosage range: roughly {optimal_dose-20:.0f}-{optimal_dose+20:.0f} kg/ha "
      f"(near the peak, avoiding both under- and over-application)")
```

### Output



## Q7

# STL Decomposition — Ticket Sales

### Code (R)

```
# Simulated dataset: monthly airline ticket sales over 5 years (60 months),
# with strong holiday-season seasonality (Nov-Dec-Jan peaks) layered on a
# slow underlying DECLINE that seasonality tends to mask.
set.seed(7)
n_months <- 60
month_index <- 1:n_months
trend_component <- 50000 - 100 * month_index          # slow decline
seasonal_component <- 8000 * sin(2 * pi * month_index / 12 + 1.2) # yearly holiday cycle
noise <- rnorm(n_months, 0, 1200)
sales <- trend_component + seasonal_component + noise

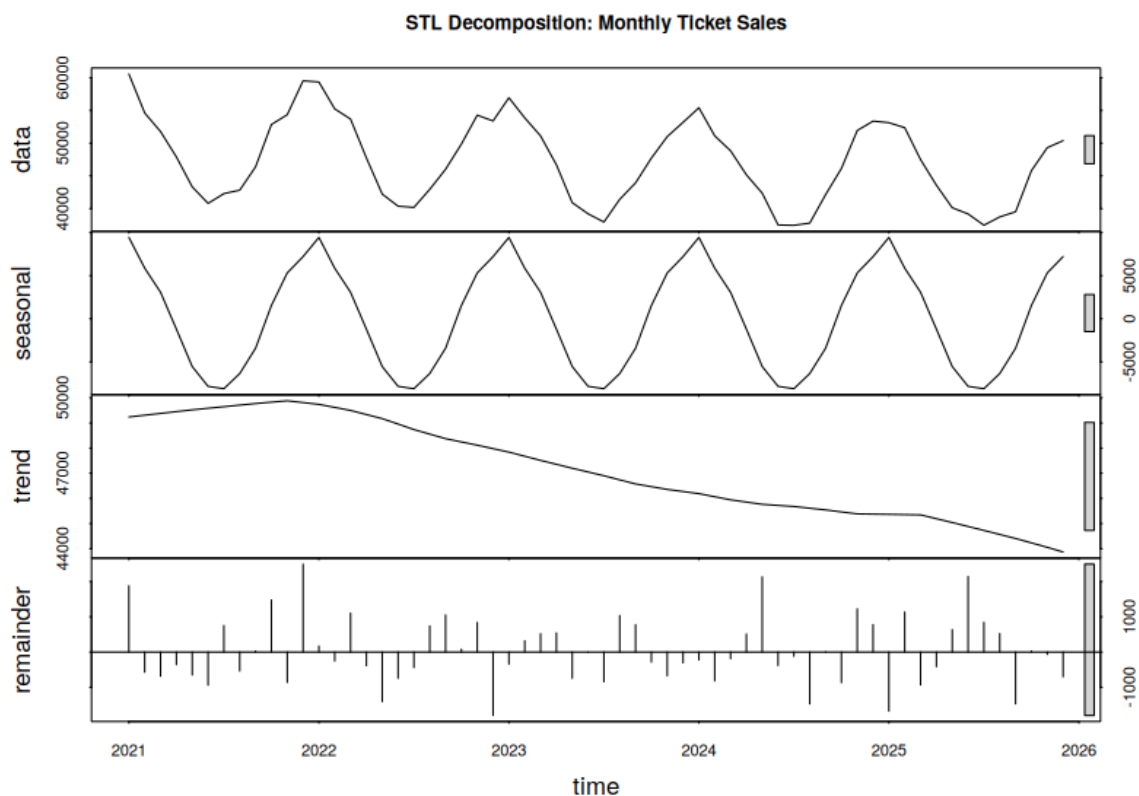
sales_ts <- ts(sales, start=c(2021, 1), frequency=12)

stl_result <- stl(sales_ts, s.window="periodic")
plot(stl_result, main="STL Decomposition: Monthly Ticket Sales")

trend_vals <- stl_result$time.series[, "trend"]
cat(sprintf("Trend value at start: %.0f\n", trend_vals[1]))
cat(sprintf("Trend value at end:   %.0f\n", trend_vals[n_months]))
cat(sprintf("Net change in trend over 5 years: %.0f\n", trend_vals[n_months] - trend_vals[1]))

# Interpretation: once STL removes the seasonal holiday cycle, the
# remaining TREND component shows a clear, steady decline across the full
# 5-year window - confirming the revenue team's suspicion that a real
# underlying decline exists and was being masked by strong seasonal peaks
# in the raw (non-decomposed) sales series.
```

### Output



## Q8

# STL Decomposition — Weekly Active Users

---

### Code (Python)

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from statsmodels.tsa.seasonal import STL

# Simulated dataset: weekly active users over 2 years (104 weeks), with a
# recurring annual seasonal dip (e.g. summer viewing slump) but a FLAT
# underlying trend - the "decline" is mostly seasonal, not structural.
np.random.seed(8)
n_weeks = 104
weeks = pd.date_range("2024-01-07", periods=n_weeks, freq='W')
trend_component = 500000 + 300 * np.sin(np.linspace(0, np.pi, n_weeks)) # roughly flat/mild
seasonal_component = 40000 * np.sin(2 * np.pi * np.arange(n_weeks) / 52 + 2)
noise = np.random.normal(0, 8000, n_weeks)
users = trend_component + seasonal_component + noise

series = pd.Series(users, index=weeks)
stl = STL(series, period=52, robust=True)
result = stl.fit()

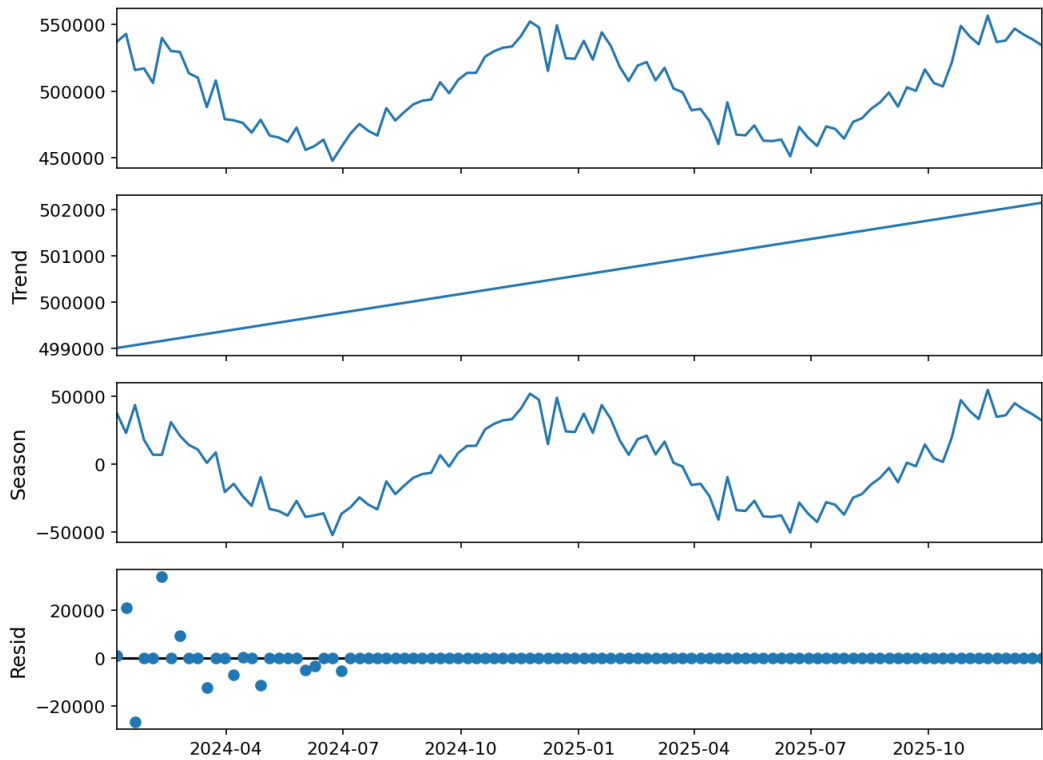
fig = result.plot()
fig.set_size_inches(9, 7)
fig.suptitle("STL Decomposition: Weekly Active Users", y=1.01)
plt.tight_layout()
plt.show()

trend = result.trend
print(f"Trend at start: {trend.iloc[0]:.0f}")
print(f"Trend at end: {trend.iloc[-1]:.0f}")
print(f"Net change in trend over 2 years: {trend.iloc[-1] - trend.iloc[0]:.0f}")
print(f"Trend range (max-min): {trend.max() - trend.min():.0f}")

"""
Interpretation for the product team: after STL removes the recurring
annual seasonal cycle, the TREND component stays essentially flat/mildly
fluctuating rather than showing a sustained downward slope - meaning the
recent dips in weekly active users are best explained by NORMAL SEASONAL
VIEWING HABITS, not a genuine underlying decline in the user base. The
product team should not treat the dip as an alarm signal on its own; it's
consistent with the same seasonal pattern recurring year over year.
"""
```

### Output

STL Decomposition: Weekly Active Users



## Q9

# 7-Day & 30-Day Moving Average Crossover

---

### Code (R)

```
library(ggplot2)
library(zoo)

# Simulated dataset: a volatile stock's daily closing price over 1 trading
# year (250 days) - a random walk with a regime change partway through
# (declining then recovering), to produce a genuine MA crossover.
set.seed(9)
n_days <- 250
dates <- seq(as.Date("2025-01-01"), by="day", length.out=n_days)
drift <- c(rep(-0.15, 120), rep(0.35, n_days - 120)) # decline then rally
price <- 150 + cumsum(drift + rnorm(n_days, 0, 2.2))
df <- data.frame(Date=dates, Price=price)

df$MA7 <- rollmean(df$Price, k=7, fill=NA, align="right")
df$MA30 <- rollmean(df$Price, k=30, fill=NA, align="right")

p <- ggplot(df, aes(x=Date)) +
  geom_line(aes(y=Price, color="Raw Price"), alpha=0.4) +
  geom_line(aes(y=MA7, color="7-day MA"), linewidth=0.8) +
  geom_line(aes(y=MA30, color="30-day MA"), linewidth=0.8) +
  labs(title="Stock Price with 7-day and 30-day Moving Averages",
       x="Date", y="Price ($)", color="") +
  theme_minimal()
print(p)

# Detect crossover points: where MA7 - MA30 changes sign
diff_ma <- df$MA7 - df$MA30
sign_change <- which(diff(sign(diff_ma)) != 0)
crossover_dates <- df$Date[sign_change + 1]
cat("Crossover dates (MA7 crosses MA30):\n")
print(crossover_dates)

# Interpretation: when the short-term (7-day) average crosses ABOVE the
# long-term (30-day) average (a "golden cross"), it signals that recent
# momentum is turning more positive than the longer-term trend - often
# read as a buy signal. A cross BELOW ("death cross") signals recent
# momentum weakening relative to the longer trend - often read as a sell
# signal. The crossover date(s) above mark where this stock's short-term
# momentum shifted relative to its longer-term trend.
```

### Output

Stock Price with 7-day and 30-day Moving Averages



## Q10

# Detrending Quarterly GDP

---

### Code (Python)

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Simulated dataset: quarterly GDP (in $ billion) over 20 years (80
# quarters), with a strong long-term growth trend plus business-cycle-like
# fluctuations (a recession dip and a recovery bump) riding on top.
np.random.seed(10)
n_quarters = 80
quarters = pd.period_range("2006Q1", periods=n_quarters, freq='Q')
trend_component = 500 + 8 * np.arange(n_quarters)
business_cycle = 30 * np.sin(2 * np.pi * np.arange(n_quarters) / 24) # ~6-year cycle
recession_dip = np.zeros(n_quarters)
recession_dip[35:41] = -40 * np.sin(np.linspace(0, np.pi, 6)) # a modeled recession
noise = np.random.normal(0, 6, n_quarters)
gdp = trend_component + business_cycle + recession_dip + noise

series = pd.Series(gdp, index=quarters.to_timestamp())

# Method 1: differencing (removes trend by taking quarter-over-quarter change)
gdp_diff = series.diff().dropna()

# Method 2: regression-based detrending (fit a linear trend, subtract it)
x = np.arange(n_quarters)
coeffs = np.polyfit(x, gdp, 1)
linear_trend = np.polyval(coeffs, x)
gdp_detrended_regression = gdp - linear_trend

fig, axes = plt.subplots(3, 1, figsize=(9, 9), sharex=False)

axes[0].plot(series.index, series.values, color='steelblue')
axes[0].plot(series.index, linear_trend, color='red', linestyle='--', label='Fitted linear trend')
axes[0].set_title("Raw Quarterly GDP with Fitted Trend")
axes[0].legend()

axes[1].plot(gdp_diff.index, gdp_diff.values, color='darkorange')
axes[1].axhline(0, color='gray', linewidth=0.8)
axes[1].set_title("Detrended via Differencing (Quarter-over-Quarter Change)")

axes[2].plot(series.index, gdp_detrended_regression, color='seagreen')
axes[2].axhline(0, color='gray', linewidth=0.8)
axes[2].set_title("Detrended via Regression (GDP minus Fitted Linear Trend)")

plt.tight_layout()
plt.show()

print(f"Fitted linear trend: GDP = {coeffs[1]:.1f} + {coeffs[0]:.2f} * quarter_index")
print(f"Largest negative deviation (regression-detrended): "
      f"{gdp_detrended_regression.min():.1f} at {series.index[np.argmin(gdp_detrended_regression)].date()}")

"""
Interpretation: once the steady long-term growth trend is removed (by
either method), the remaining fluctuations reveal the underlying
BUSINESS-CYCLE pattern - a smooth multi-year rise-and-fall oscillation,
with one clearly deeper dip corresponding to the simulated recession
period. This is exactly the kind of cyclical detail the raw GDP series
(dominated visually by its strong upward trend) would otherwise hide -
detrending isolates the business-cycle signal the economist actually
wants to study in isolation.
"""
```

### Output

